COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 04

Debugging, testing and collaboration

Four expanded lessons with worked examples, practical assignments and five-question quizzes. Use the main workbook for this module’s project and additional exercises.

4 lessons · 4 demonstrations · 20 lesson questions

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

Contents

Use the linked contents or PDF bookmarks to navigate.

Module 04: Debugging, testing and collaboration 3
Lesson 04.1: Debugging as an experiment 3
Lesson 04.2: Unit, integration and property tests 6
Lesson 04.3: Git, branches and recoverable changes 9
Lesson 04.4: Review, documentation and responsible AI help 12

Module 04: Debugging, testing and collaboration

Prerequisites: Modules 01-03.

Integration project: A reviewed, tested local release. The full brief and review are in the Lab Workbook.

Lesson 04.1: Debugging as an experiment

Learning outcome: Find a defect by narrowing evidence rather than making random edits.

A bug report begins with a violated expectation

“The health system is broken” is not enough to reproduce a defect. A useful report states the starting state, input sequence, expected result, actual result and environment. For a single hit of damage 3 applied to health 10, the contract expects health 7. Observing health 4 suggests a different operation occurred; it does not yet prove where the extra subtraction happened.

Keep the reproduction small. One character and one hit are better than a complicated match when they show the same failure. Remove irrelevant input, rendering and timing while preserving the difference. A minimal reproduction is an experiment whose outcome distinguishes correct behavior from the defect.

Find the first divergence

Instrument values at a few boundaries: before the event, after the simulation update and before drawing. If health is already 4 after simulation, changing the display format cannot fix the first incorrect state. If simulation reports 7 but drawing later changes it to 4, inspect whether the presentation layer is applying damage a second time.

You can use print tracing or a debugger breakpoint. Both are ways to observe execution, not substitutes for a hypothesis. Choose a location because it can distinguish competing explanations. Do not add hundreds of prints and hope the volume of output reveals the cause by itself.

Form a falsifiable hypothesis

A useful hypothesis is “damage is subtracted in both the update handler and the render handler.” It predicts that drawing twice after one hit will reduce health again. Another hypothesis is “the queued hit appears twice,” which predicts two event records before drawing. These explanations need different repairs, even if both produce the same final health.

Change one relevant factor at a time and record the result. Randomly moving statements until the symptom disappears can introduce a new defect or hide the original one for the observed input. A repair is more convincing when the failing test first demonstrates the issue and then passes after the smallest justified change.

Regression tests preserve what you learned

A regression test encodes the behavior that failed so a future change can reveal its return. For the pure damage rule, assert that one event applied to health 10 with damage 3 returns 7. Also test lethal damage, zero damage and already-zero health. Invalid negative damage is not healing under this contract; it must be rejected.

A focused test is not proof that every event is dispatched once in the complete game. That broader risk needs an integration test showing that one command produces one update. Separate the narrow arithmetic contract from the wiring contract. A correct function called twice can still make a broken game.

Do not debug by suppressing all evidence

Catching every exception and continuing with an arbitrary value can make a program look stable while corrupting its state. Removing an assertion because it fails may silence the detector instead of repairing the violation. The appropriate question is whether the contract, implementation or test is wrong, and the answer requires independent reasoning from the specification.

Keep expected and actual results visible. Preserve the failing input with the fix. Explain why the modified code corrects the rule rather than merely matching one result. A reviewer should be able to introduce the old defect in a disposable copy and observe the regression test fail again.

Learn from the failure without inflating the claim

The included mutation demonstration intentionally doubles damage and verifies that one focused assertion rejects it. This shows the test can detect that known mutation. It does not establish that the test suite finds every possible defect. Be equally precise in your own lab record: name the seeded bug, the detecting test and the boundary of what was checked.

Worked example

Predict before running `examples/example_04_1.py`.

```
def buggy_damage(health, damage):
    health = max(0, health - damage)
    health = max(0, health - damage)  # Deliberate duplicate update.
    return health

starting_health = 10
one_hit = 3
expected_health = 7
actual_health = buggy_damage(starting_health, one_hit)
print("expected:", expected_health)
print("actual:", actual_health)
print("defect observed:", actual_health != expected_health)
```

This example is deliberately wrong and labeled as such. The second subtraction is another update, not display work. Trace the intermediate value after the first subtraction to identify the earliest statement that makes the result disagree with the one-event contract. Do not treat running this demonstration as completing the repair lab.

Expected demonstration output

```
expected: 7
actual: 4
defect observed: True
```

Practical work and pass evidence

Introduce a double-damage bug into a disposable copy of your simulator. Write a minimal report with health 10, damage 3, expected 7 and actual behavior. Trace or break at the first incorrect update. Add a failing test before repairing the copied simulator.

Implement `apply_damage` in `labs/student/lab_04_1.py`: nonnegative integer inputs, one subtraction and a zero floor. Demonstrate that the repaired version passes, and that temporarily restoring the known bug makes a relevant test fail. The supplied `tools/mutation_demo.py` illustrates this method; your own bug report and regression test remain required.

Changed-case challenge

Construct an integration bug where a correct damage function is called twice. Explain why fixing the arithmetic function would be the wrong repair.

Companion source study

[S08] Python: Errors and Exceptions; [S12] Python unittest library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What information makes a bug report actionable?
2. Why inspect the earliest wrong state?
3. What is a regression test?
4. A correct damage function returns 7 for health 10 and damage 3, but the whole game reaches 4. What layer should you investigate next?
5. Why should a regression test fail before the fix is applied?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 04.2: Unit, integration and property tests

Learning outcome: Choose tests that challenge the contract instead of repeating the implementation.

Test a contract, not a copy of the code

An automated test sets up a known case, invokes the operation and compares the outcome with an expectation. The expected result should come from a rule, derivation or independent reference. If the test calculates its expected value by copying the implementation, the same error can exist on both sides and remain invisible.

For a one-dimensional board of size 4, valid positions are 0 through 3. Moving right from 3 is rejected under our whole-move rejection policy, so the result remains 3. An expected result derived from the written coordinate rule is more independent than another copy of the same potentially incorrect comparison.

Unit, integration and property-style checks

A unit test targets a small operation, such as spending resources. An integration test checks cooperating parts, such as encoding a state, writing it, reading it and decoding it. A property-style check tests a relation across many inputs: every accepted movement result remains within the board, for example.

The simple property loops in this package enumerate bounded input sets using ordinary Python. They are not exhaustive proofs over all integers and they do not require a third-party property-testing library. Record the tested domain. A broad-looking loop can still omit the exact boundary that matters.

Properties need more than one dimension

An implementation that always returns the starting position stays in bounds, but fails to move when it should. An implementation that always returns zero also stays in bounds for many tests. Therefore combine safety properties with specific expected transitions and progress requirements. Correctness is not one convenient invariant; it is satisfaction of the relevant contract.

Similarly, a save round trip alone may allow an encoder and decoder to share an incorrect schema. Include independent known fixtures and rejection cases. Check that loading malformed data fails without modifying the previous valid state. Separate evidence for ordinary operation from evidence for failure behavior.

Make tests isolated and repeatable

Each test should establish the state it needs rather than depend on the previous test's side effects. Use fresh lists or dictionaries for mutable data. File tests should use a temporary directory or an explicitly disposable workspace rather than overwrite a personal file. Clean up only files the test created.

Use descriptive test names such as `test_right_edge_rejects_move` rather than `test_7`. When a failure occurs, the name should indicate the violated rule. Keep assertions small enough that the failure report identifies which property disagreed. A passing suite is useful evidence, but its value depends on the cases and the independence of their expectations.

Learn the runner without confusing it with your work

Python's `unittest` framework discovers and executes test methods. The supplied checker runs package-authored contract tests against student or reference implementations. Reference mode verifies the supplied answers; it does not check your implementation and cannot certify learning. Student mode points at the editable starter files, which intentionally fail until completed.

You must also write at least twelve tests spanning motion, resource spending and save/load. Do not submit the supplied tests unchanged as learner-authored work. Use them as examples of the interface, then challenge it with additional boundaries and failure conditions.

Test the tests with a known mutation

In a disposable copy, replace `< size` with `<= size`, or subtract damage twice. A relevant test should fail. If it does not, the suite has a gap for that mutation or is not executing the code you think it is. Restore the correct implementation after the experiment and record both outcomes. Do not leave intentional bugs in the version you later package.

Worked example

Predict before running `examples/example_04_2.py`.

```
def bounded_move(position, delta, size):
    candidate = position + delta
    if 0 <= candidate < size:
        return candidate
    return position

cases = [(2, 1, 4, 3), (3, 1, 4, 3), (0, -1, 4, 0)]
for position, delta, size, expected in cases:
    actual = bounded_move(position, delta, size)
    assert actual == expected
print("three independent boundary cases passed")

for position in range(4):
    for delta in (-1, 0, 1):
        assert 0 <= bounded_move(position, delta, 4) < 4
print("twelve bounded-input property cases passed")
```

The three explicit cases test a successful move and both edges. The property loop tests twelve additional input combinations, but some overlap the explicit cases; do not report them as fifteen unique input cases without checking. Each assertion uses ordinary Python assertion semantics; the fuller package checker uses unittest assertions and should be run without intentionally disabling checks.

Expected demonstration output

```
three independent boundary cases passed
twelve bounded-input property cases passed
```

Practical work and pass evidence

Implement `bounded_move` in `labs/student/lab_04_2.py` with the documented whole-move rejection policy. The starting size must be positive and the starting position inside the board. Then write at least twelve learner-owned tests in `workspace/test_my_rules.py` across motion, spending and save/load, including at least one bounded property loop.

Record each test's requirement and independently derived expected result. Introduce an off-by-one or double-update mutation in a disposable copy and show a named test detects it. Restore the correct code. A complete submission includes tests, a successful run and the mutation experiment, not only the supplied checker output.

Changed-case challenge

Write a deliberately useless implementation that never moves. Identify which property tests it passes and which expected-transition test rejects it.

Companion source study

[S12] Python unittest library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. How does an integration test differ from a unit test?
2. Why is copying the implementation into the test oracle weak?
3. What does a test suite passing establish?
4. Why is a bounds-only property insufficient for a movement function?
5. Does `--reference` on the supplied checker validate your edited student files?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 04.3: Git, branches and recoverable changes

Learning outcome: Use small commits to preserve reasoning and collaborate safely.

History should preserve reasoning

Version control records meaningful states of a project and the changes connecting them. A commit is most useful when it represents a focused change with a message explaining its purpose. “Stuff” or “final final” does not tell a future reviewer why a rule changed. Commit after a coherent improvement with tests and evidence, not merely after a large amount of typing.

A local Git repository is enough for this lesson. A hosting account, public upload and remote push are not required. Keep personal credentials, private documents, real player data and generated temporary files out of the repository. Review the staged content before making a commit rather than assuming every file in the directory belongs in project history.

Working tree, index and commit

The working tree contains the files you are editing. The staging area, also called the index, selects the content for the next commit. The commit records that staged state. Editing a file after staging it does not automatically stage the later edit. Inspect both the unstaged difference and the staged difference when the distinction matters.

Use `git status` to see the current situation, `git diff` for unstaged changes and `git diff --staged` for staged changes. Use `git add` with explicit file paths while learning. A broad add command can include unrelated material that happened to be present. A `.gitignore` rule helps prevent accidental tracking of future generated files, but it does not erase already committed content.

Branches isolate a line of work

A branch gives a name to a line of development. Start from a known baseline, create a feature branch, make a focused change and test it. Merging incorporates the branch's work into the destination branch. A fast-forward merge is possible when the destination has not diverged from the branch's ancestor; it need not create an extra merge commit.

A branch is not a separate backup of every external file on your computer. Git tracks the repository's content and history under its rules. Keep ordinary backups where appropriate, especially before learning destructive operations. This lesson deliberately avoids force pushes and hard-reset cleanup as a normal recovery workflow.

Revert a mistake without rewriting the lesson

`git revert` creates a new commit that reverses a selected earlier change. The mistaken commit remains visible in history, along with the correction. This is useful for demonstrating why the change was wrong and how it was repaired. More complex histories can produce conflicts during a revert; the beginner exercise chooses a small nonconflicting history.

Do the exercise in a disposable new repository, not an employer repository or an important existing project. The supplied sandbox tool creates its own temporary directory and uses command-scoped identity, leaving global configuration alone. Its automated demonstration is a reference; you must still perform and explain your own three-commit and branch exercise.

A concrete safe sequence

In your new practice folder, initialize a repository with a main branch, add a small rules file and make a baseline commit. Create a `boundary-fix` branch, add an explicit zero-health rule, commit and merge it. Then make an intentionally wrong damage change and revert that new commit. Inspect the final file and log. The final rules should match the intended pre-mistake version while the history retains the mistake and revert.

Configure author identity only according to your own environment and privacy choices. Do not copy someone else's name or email into commits. Use local repository settings for a practice repository when needed, and inspect their scope. The companion worksheet supplies the shell commands and asks you to record what changed at each stage.

Worked example

Predict before running `examples/example_04_3.py`.

```
from pathlib import Path
import runpy

package_root = Path(__file__).resolve().parents[1]
print("Running the isolated Git demonstration.")
runpy.run_path(str(package_root / "tools" / "git_sandbox.py"),
               run_name="__main__")
```

This Python wrapper launches the supplied temporary-repository demonstration. It does not run commands against your current Git project. Commit IDs in its output vary because commit metadata and timing vary. The expected evidence is a successful branch/merge/revert sequence and restored rules, not a specific hash copied from a screenshot. Git must be installed for this example.

Expected demonstration output

```
Temporary repository only; no existing course files changed.
[commit graph; commit IDs vary]
Restored rules:
damage=3
health_min=0
PASS: branch, merge, revert and clean working tree.
```

Practical work and pass evidence

Use the Git worksheet in `templates/GIT_LAB.md` inside a new disposable practice directory. Make three focused commits, implement a change on a branch, merge it and revert an intentionally bad commit. Before committing, inspect staged differences. Save the log, final rules, commands and an explanation of the staging area in `workspace/04_3_git_evidence.md`.

Run the supplied sandbox demonstration as a comparison, not as a substitute for your own exercise. Pass by identifying the exact content of each commit, explaining why the revert adds history and showing that your working tree is clean after completion.

Changed-case challenge

Stage a file, edit it again, then compare git diff with git diff --staged. Explain which version would enter the next commit.

Companion source study

[S13] Pro Git: Recording Changes to the Repository; [S16] Git revert documentation. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What does the staging area control?
2. Why should secrets not be committed even if later deleted?
3. What makes a useful commit message?
4. You stage version A of a file, edit it to version B, and commit without staging again. Which content is committed?
5. Does reverting a mistaken commit normally remove that commit from the project history?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 04.4: Review, documentation and responsible AI help

Learning outcome: Explain a change and verify assistance without surrendering understanding.

Review starts with the requirement

A reviewer cannot judge a change solely from whether the code looks tidy. Ask what behavior is required, which cases changed, which assumptions are trusted and what evidence supports the claim. A ten-line function can violate an important boundary even when its variable names are clear.

For a purchasing rule, determine whether exact affordability is allowed, what zero cost means and how negative values are handled. A review comment such as “use a shorter expression” is secondary if the comparison rejects a purchase costing exactly the available balance. Prioritize correctness and clear contracts before stylistic preferences.

Read a diff as a change in behavior

A diff shows what changed between versions. Trace the changed lines using at least one normal input, one boundary input and one invalid input. Ask whether the tests distinguish the old behavior from the intended new behavior. A test added with an expectation copied from the changed code is weak evidence if nobody checked it against the requirement.

Review side effects too. A function that previously only returned a result might now write a file or print a message. That may break callers even when the numeric output is unchanged. The interface includes effects and failure behavior, not only parameter names.

Write instructions another person can execute

A README should identify the required interpreter, the working directory, exact commands, expected success signal, controls, test command and known limitations. A person starting from a clean extraction should not need your memory of an unrecorded setup step. “Run the project” is not an executable instruction.

Test the README by following it in a fresh directory. Keep generated state and hidden editor configuration from becoming undeclared prerequisites. Record the actual interpreter used rather than claiming compatibility with every possible environment. When a tester encounters a failure, improve the instructions or implementation based on evidence.

Assistance must not replace understanding

AI, documentation and peers can help explain an error, suggest cases or review a design. Record material assistance and verify the result. Predict the operation before accepting generated code; explain each changed line; run tests and create a changed example without following the answer. A passing test suite does not establish that you understand an implementation well enough to maintain it.

Do not upload private employer source, credentials, proprietary SDK material or another person's personal data without permission. Use a minimal non-sensitive reproduction when asking for help. The programming exercise is an opportunity to learn better boundaries, not a reason to relax them.

Separate what the reviewer knows from what is assumed

A reviewer may verify a code path by reading it, run selected tests or reproduce the whole package. These are different evidence levels. State which happened. “Reviewed” should not imply that every supported platform was physically tested when only a short diff was inspected.

Likewise, acknowledge limits in your own handoff. You may have run the program on one machine and asked one person to try it. That is a useful start, not a comprehensive usability study. The purpose is to build habits of precise claims and repeatable evidence.

Practice a live change

After reviewing, close the reference and implement a related variation, such as changing a threshold from 5 to 6 or adding a zero-cost case. Predict which tests should change and which should not. If you cannot explain the effect without copying a solution, return to the underlying lesson. A review is successful when it improves both the artifact and your ability to reason about it.

Worked example

Predict before running `examples/example_04_4.py`.

```
def proposed_purchase(balance, cost):
    # Deliberate review candidate: inspect the boundary comparison.
    if balance > cost:
        return balance - cost
    return balance

for balance, cost in ((6, 5), (5, 5), (4, 5)):
    print("balance/cost/result:", balance, cost,
          proposed_purchase(balance, cost))
print("Review question: should exact affordability be accepted?")
```

The candidate is intentionally not endorsed as a correct purchasing rule. Its normal case works, which can distract from the exact-affordability boundary. Compare the result with the original nonnegative spending contract. Also note that the demonstration does not validate negative inputs; a complete review must identify that missing boundary separately.

Expected demonstration output

```
balance/cost/result: 6 5 1
balance/cost/result: 5 5 5
balance/cost/result: 4 5 4
Review question: should exact affordability be accepted?
```

Practical work and pass evidence

Review a roughly ten-line change using `templates/CODE_REVIEW.md`. Address requirement, correctness, boundaries, side effects, clarity and tests. Produce a README for your simulator that another person can follow from a clean folder. Record assistance and provenance in the template.

Perform one small related change without copying the reference, predict its effects and update the tests. This is an evidence-based lab; there is no automatic function checker for it. Pass by explaining every changed line and by recording a clean-start result from another person or a clearly labeled self-review when independent review is not yet available.

Changed-case challenge

Review a change that moves input() into a previously pure function. Explain the interface regression even when normal interactive play still appears to work.

Companion source study

[S01] Harvard CS50 Introduction to Programming with Python; [S12] Python unittest library; [S13] Pro Git: Recording Changes to the Repository. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What should a review criticize?
2. Why is compiling AI-generated code insufficient validation?
3. What demonstrates understanding beyond copying a solution?
4. What is the first requirement to clarify when reviewing the purchase candidate with balance=5 and cost=5?
5. Why is “the AI wrote it and all supplied tests pass” insufficient for this course's review check?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.